

Chapters 4 and 5, Before You Read

The big ideas in plain language, plus what to focus on in a long chapter

Chapters 4 and 5 cover a lot of ground: why memory comes in layers, why that works at all, and the handful of design decisions every cache on earth is built from. This document does two things. Part 1 walks through the big ideas in plain language, the same ideas the slides cover, with a little more room to explain the “why” behind each one. Part 2 narrows the two chapters down to what is worth a close read versus a quick skim, so the length of the chapters does not get in the way of the handful of ideas that matter most for class. In class, we will pick several of these up directly on a live Linux machine, so nothing here needs to be memorized ahead of time, just understood well enough to recognize when we see it.

1. The Big Ideas, In Plain Language

What Makes One Memory Different From Another

Whenever engineers compare two memory technologies, whether that is a CPU register against a hard disk, or one cache design against another, they are really comparing four things. **Location** asks whether the memory sits inside the processor or outside it; note that “inside” includes both registers and cache, not only main memory, since cache is itself a form of internal memory. **Capacity** asks how much it holds. **Access method** asks how you get to a given piece of data: sequentially, like tape, where you pass every record in between; directly, like disk, where you jump to the neighborhood and then search a little; randomly, like RAM, where any address costs the same regardless of what was accessed before; or associatively, where a piece of hardware is asked “do you have this value,” rather than “give me what is at this address.” **Performance** asks about access time, cycle time, and transfer rate, and this is the one a memory hierarchy exists to improve.

Why Memory Comes in Layers

No single technology gives you large, cheap, and fast all at once. Every technology available trades one of these for another: faster costs more per bit, bigger costs less per bit but takes longer to reach. Rather than pick one point on that trade-off, real systems stack several technologies: registers, then cache, then main memory, then solid-state or magnetic storage, then tape for cold offline storage. One detail worth knowing that is easy to miss on a diagram is who is actually in charge of each layer. Registers are managed by the compiler, which decides what values live there. Cache is managed entirely by hardware, invisible to any program running on the machine. Main memory is managed by the operating system. Secondary storage is managed jointly by the operating system and by you, the user, since you decide what files to keep. The layers are not only about speed, they are also about who gets to decide what lives where.

Why the Hierarchy Actually Works: The Principle of Locality

None of the above would help if programs touched memory in a truly random pattern; a hierarchy only pays off if whatever was used recently is likely to be used again soon. This is the principle of locality, and it holds for an intuitive reason: real programs spend most of their time inside loops

and subroutines, re-executing a small set of instructions over and over (temporal locality), and they often walk through arrays and data structures in order, touching nearby addresses one after another (spatial locality). This is not a lucky coincidence, it is close to unavoidable given how loops, function calls, and arrays work in ordinary code. Without it, a cache would just be an expensive way to slow a computer down; every layer of the hierarchy is a bet that locality holds, and that bet almost always pays off.

Every Cache Is Built From the Same Handful of Decisions

Once you know six things about a cache, you can describe practically any cache ever built, from a 1970s mainframe to the chip in a phone: how addresses are represented (cache addresses), how big it is (size), how a block of main memory is placed inside it (mapping function), what gets thrown out when it is full (replacement algorithm), how it stays consistent with main memory (write policy), and how many levels there are, and whether each level is split or unified. Table 5.1 in the book lists exactly these. The rest of Chapter 5 is really just working through each of the six in turn.

How Big Is a Word, Really?

The rest of the chapter throws around a handful of size words, bit, byte, word, cache line, page, as if everyone already agrees on them, so it is worth pinning them down once. A **bit** is a single binary digit, 0 or 1. A **byte** is 8 bits, enough to represent one character of text. A **word** is the processor's natural chunk of data, the size it is built to move and compute with in one step, typically 4 bytes on a 32-bit machine or 8 bytes on a 64-bit one; this is the same “word” Chapter 4 uses when it talks about what a register holds. A **cache line** (or block) is typically 64 bytes on current hardware, the amount of data moved in one trip between cache and main memory, and is made up of 16 words on a 32-bit machine. A **page** is typically 4 KB, the unit the operating system uses when it moves data between main memory and disk. Each one is simply the previous one multiplied up by a factor that happens to be convenient for the hardware, and none of them are interchangeable even though it is tempting to use “word” loosely to mean “some small chunk of data.” Line size in particular is worth holding onto, it is the number that turns up directly in the walkthrough as `coherency_line_size`.

Placing a Block: The Three Mapping Functions

Main memory addressing and cache addressing are not the same kind of thing, even though the same address value is involved in both. In main memory, an address is a guarantee: it names one unique, permanent location, and whatever is asked for is simply there, no verification needed. A cache is far smaller than main memory, so it cannot give every possible address a permanent home of its own; many different blocks are forced to take turns sharing the same cache line over time. That means a cache address cannot work as a guarantee the way a memory address does. It works as a hypothesis: part of the address narrows the search down to a candidate location, and a second part, the tag, is then checked against what is actually sitting there to confirm the guess was right. Every mapping function below is really a different answer to how much of that work is handled by narrowing the location versus by verifying identity.

When a block of main memory needs to go into the cache, three approaches exist. **Direct mapping** sends every block to exactly one possible line, computed with a simple remainder (block number mod number of lines). It is cheap to check, but if a program happens to alternate between two blocks that map to the same line, they will fight for it constantly, a problem called thrashing. **Associative mapping** allows a block to go in any line at all, which avoids thrashing, but the hardware has to

compare the tag of every single line at once to find a match, which is expensive. **Set-associative mapping** splits the difference: the cache is divided into sets, each holding a handful of lines, and a block maps to exactly one set but can occupy any line within it. Almost every processor built in the last few decades uses this compromise.

A useful detail for reading a real cache's numbers: the address the processor sends is split into three fields. A few bits at the end, the offset, pick out one byte within a block. The next few bits, sometimes called the line or index bits, pick out which set to look in. The remaining bits, the tag, are compared against what is stored in each line of that set to confirm which specific block is actually sitting there. This tag comparison is the piece with no equivalent at all in ordinary memory addressing: it exists only because the cache is reused, oversubscribed storage, so an index alone can never be trusted the way a full memory address can be. Knowing this split makes it possible to look at a real processor's cache and work out exactly how many sets it has, which we will do together in class.

Keeping Cache Honest, and Why There's More Than One Level

When a program writes to a memory location that happens to be cached, the cache and main memory temporarily disagree about that value, and something has to reconcile them. **Write-through** updates both immediately: simple and always consistent, but every write now costs a trip to main memory. **Write-back** updates only the cache and marks the line dirty, writing it back to main memory only when that line is eventually evicted: far less traffic to main memory, at the cost of more bookkeeping and a period where main memory is quietly out of date.

As chips gained enough transistor budget to put cache directly on the same die as the processor, a single on-chip cache (L1) was no longer the end of the story: it is small precisely because small caches are fast, which leaves room for a second, larger, slightly slower cache (L2) to catch what L1 misses, and often a third (L3) shared across cores to catch what L2 misses, before finally falling back to main memory. On a typical modern chip, L1 lives on each individual core and is private to it, L2 is usually private to each core as well in current designs (though older chips sometimes shared it across a small cluster), and L3 sits on the chip but is shared across all the cores, larger and slower than L1 or L2 but still far faster than reaching out to main memory.

2. How to Read the Chapters

Both chapters are long, and not every section carries equal weight for what we do together in class. Below, each chapter is split into three groups: read closely, skim for the general idea, and skip for now. "Skip for now" does not mean permanently skip, it means these sections matter less for this particular class session, not that they are unimportant to the course as a whole.

Chapter 4: The Memory Hierarchy, Locality and Performance

Read closely

- Principle of Locality, both temporal and spatial. This is the assumption the entire chapter, and Chapter 5, rest on, and it is exactly what our live timing demo in class will measure directly.
- The Memory Hierarchy itself (Figure 4.6) and the reasoning behind it: the trade-off among capacity, access time, and cost, and why the answer is to stack several technologies rather than pick one.

- Characteristics of Memory Systems (Table 4.1): location, capacity, access method, and performance. This is the vocabulary the rest of the chapter, and Chapter 5, are written in.
- Table 4.2, Characteristics of Memory Devices. The “managed by” column is easy to skim past but is genuinely useful: it explains who is actually in control of each layer of the hierarchy.
- Operation of Two-Level Memory: the M1/M2 relationship, and the idea of a hit ratio. We will find this exact relationship at work in Linux’s own page cache in class.

Skim for the idea

- Design Principles for a Memory Hierarchy (locality, inclusion, coherence). You need to recognize these three names, not derive them.
- Figure 4.9, the sizes of the chunks of data exchanged between levels (words, cache blocks, pages). The general idea that different levels move data in different-sized units is enough.

Skip for now

- Figures 4.7, 4.8, and 4.11 through 4.14, the cost and access-time curves and the hit-ratio derivations. The conclusions matter more than the graphs; we will not be deriving these formulas in class.
- Figure 4.10, the IBM z13 case study. An interesting real example, not required background for this session.

Chapter 5: Cache Memory

Read closely

- Cache Memory Principles: block, line, tag, and line size. Small vocabulary words that the rest of the chapter uses constantly.
- Figure 5.3, the cache read operation (hit versus miss). This is the single most important diagram in the chapter.
- Table 5.1, Elements of Cache Design. This table is effectively the chapter’s table of contents: cache addresses, size, mapping function, replacement algorithm, write policy, line size, number of caches.
- Cache Size, and why bigger is not automatically better.
- Table 5.3 and the discussion of direct, associative, and set-associative mapping (Figures 5.6, 5.7, 5.10). We will use this directly to read our own processor’s cache settings in class.
- Replacement Algorithms: LRU, FIFO, and LFU, and why LRU tends to win on simplicity.
- Write Policy: write-through versus write-back, and what a dirty bit is for.
- Multilevel Caches, and Unified versus Split Caches. This explains why your own machine will show a split L1 but a unified L2 and L3.

Skim for the idea

- Cache Addresses: logical versus physical caches. Good to know the distinction exists; the full reasoning depends on virtual memory, which is Chapter 9.
- Content-Addressable Memory (CAM). Useful background on how associative mapping is actually built in hardware, not required to use the concept.
- Write Miss Alternatives (write allocate versus no-write allocate) and Cache Coherency. Both are real design questions, more relevant once multiple processors and multiple caches are in the picture, which is a later chapter's topic.
- Line Size. The general trade-off (too small wastes the benefit of fetching a block at all, too large wastes bandwidth on data that will not be used) is enough for now.
- Inclusion Policy (inclusive, exclusive, noninclusive). Recognize the three names; the full implications matter more once we discuss multiprocessor cache coherence.

Skip for now

- Table 5.2, historical cache sizes of specific processors, and Table 5.4, Intel Cache Evolution. Interesting history, not needed for class.
- Figures 5.8, 5.9, 5.11 through 5.15, the worked numeric examples and performance graphs for each mapping function. The concept in the main text is what we build on; these are optional reinforcement if you want extra practice.
- Figures 5.16, 5.17, and 5.18, and Table 5.5, the Pentium 4 and IBM z13 case studies. Real and interesting, but specific implementations we will not be covering directly.
- Cache Timing Model and Tables 5.6 and 5.7. This is the mathematical modeling of cache performance; we get the same intuition from measuring a real program instead.